Sonary Booster Agent (LiteLLM) - Architecture Design v2.0

LangChain + LiteLLM + Bedrock KB

> This document describes the **new** architecture. The previous Bedrock Agent approach is preserved in ARCHITECTURE.md for reference.

Table of Contents

1 LangChain + LiteLLM + Bedrock KB
2 Table of Contents
3 Glossary
4 Executive Summary
5 Current Architecture (Before)
6 Problems With Current Architecture
7 New Architecture (After)
8 Component Deep Dive
9 Data Flow Diagrams
10 Caching Strategy
11 Session and Memory Management
12 Tool Definitions
13 Security
14 Cost Estimation
15 New Project Structure
16 Frontend Integration Changes
17 Environment Variables
18 Rollback Strategy
19 Boto3 - usage
20 Future Roadmap

Glossary

Term	Definition
LLM (Large Language Model)	An AI model that understands and generates text. Examples: Claude, Gemini, GPT, Nova. The "brain" that generates responses, but has no access to your data unless you give it tools.
LLM Provider	A cloud service that hosts and serves an LLM. You send text in via API, get text back. Examples: AWS Bedrock, Google Vertex AI, Anthropic Direct API.
Token	The unit LLMs use to measure text. Roughly 1 token = 0.75 words. You're charged per token for input (what you send) and output (what the LLM generates).
Agent	An LLM combined with instructions (a prompt) and tools. The agent receives a user question, decides which tools to call, processes results, and generates a final answer.
LangChain	A Python framework for building LLM-powered applications. Provides standardized interfaces for agents, tools, retrievers, memory, and chains. Provider-neutral -- works

	with any LLM. Chosen for its large ecosystem and multi-agent support for future expansion.
ReAct Agent	A reasoning pattern used by LangChain agents: **Re**ason + **Act**. The agent thinks about what to do, acts by calling a tool, observes the result, and repeats until it has a final answer. All tool execution happens locally inside your server.
LiteLLM	A lightweight Python library that provides a unified interface to 100+ LLM providers. Its Router feature enables multi-provider fallback: if Bedrock throttles, automatically route to Anthropic Direct API using the same model.
Router (LiteLLM)	The LiteLLM component that manages load balancing and fallback across multiple LLM providers. If the primary provider (Bedrock) returns a 429/rate-limit error, the Router retries on the fallback provider (Anthropic Direct) transparently.
ChatLiteLLMRouter	The LangChain integration class that wraps a LiteLLM Router. Used as the LLM instance inside a LangChain agent. Supports tool calling, streaming, and token usage tracking.
Tool (LangChain)	A Python function decorated with `@tool` that the agent can call. The function receives parameters, executes logic (DB query, API call, etc.), and returns a string result. The agent decides when to call each tool based on the user's question.
Retriever (LangChain)	A LangChain interface for fetching relevant documents. `AmazonKnowledgeBasesRetriever` wraps Bedrock KB behind the standard Retriever interface, allowing it to be used as a tool or chain component.
Knowledge Base (KB)	A searchable store of content (articles, documents) that uses semantic similarity (meaning-based search) instead of keyword matching. AWS Bedrock KB handles chunking, embedding, and indexing automatically.
RAG (Retrieval-Augmented Generation)	A pattern where the agent first retrieves relevant content from a Knowledge Base, then uses that content as context when generating a response. Improves accuracy for domain-specific questions.
Embedding	A numerical representation (vector) of text that captures its meaning. Similar texts have similar embeddings. Used in RAG to find relevant content.
FastAPI	A Python web framework for building HTTP APIs. The "web server" that receives chat messages from the Sonary Booster app and returns responses.
ECS Fargate	AWS's serverless container service. You provide a Docker image, it runs it. No servers to manage. Similar to Google Cloud Run.
ElastiCache Redis	AWS's managed Redis service. An in-memory data store used for fast caching (brand data, categories). Survives server restarts and is shared across instances.
RDS	AWS Relational Database Service. Your MySQL/PostgreSQL database where Sonary's brand and article data lives.
S3	AWS Simple Storage Service. File storage used as the intermediary between your database and Bedrock Knowledge Base for article ingestion.

VPC Endpoint	A private connection between your VPC and an AWS service (e.g., Bedrock, S3, ECR) that avoids routing traffic through the public internet and a NAT Gateway. Reduces costs and improves security.
Session	One conversation thread between a user and the agent. Managed by the existing Sonary App Node.js service in MySQL.
TTL (Time-To-Live)	How long cached data remains valid before it must be refreshed. Example: brand data cached with 1-hour TTL.
APScheduler	Advanced Python Scheduler. A lightweight Python library for scheduling tasks (cron jobs, intervals). Used to run the daily article sync inside the ECS service process, eliminating the need for Lambda or EventBridge.
VPC	Virtual Private Cloud. A private network inside AWS where your services communicate securely.

Executive Summary

The Sonary AI Chatbot is being re-architected to solve five critical problems:

1. **429 throttling** on Google Vertex AI blocking users from getting responses
2. **No conversation context** -- agent doesn't remember what was said earlier in the same chat
3. **Sessions lost on restart** -- all conversations disappear when the server restarts
4. **Slow responses** -- cross-cloud API calls and Vertex AI latency
5. **Inaccurate responses** -- Vertex AI Search returns fuzzy results; no structured data access

The new architecture moves from **Google Cloud (Vertex AI + ADK + Cloud Run)** to **AWS (LangChain + LiteLLM + Bedrock KB + ECS Fargate)**, consolidating all services in the same AWS VPC where the RDS database already lives.

Key architectural decision: Unlike the previous v1.0 plan (which used AWS Bedrock Agent as a managed orchestration layer), this architecture uses **LangChain as the agent framework** running inside your own ECS service. This provides:

- **No vendor lock-in** -- LLM provider can be swapped via LiteLLM without code changes
- **No rate limit dependency** -- LiteLLM Router automatically falls back across providers
- **Local tool execution** -- tools run in-process, no Return Control round-trips
- **Future-proof** -- LangChain supports multi-agent patterns (LangGraph) for future features

---

Current Architecture (Before)

```
Sonary Booster App (Frontend)
         |
         v
Sonary App Node.js (ECS)
   ┌──────────────────────────────────┐
   │  Ai.service.ts                   │
   │  - Manages sessions (UsersAiSessions)  │
   │  - Saves chat history (ChatHistory)    │
   │  - Calls AI bot via HTTP POST          │
   │  - Streams SSE responses to frontend   │
   └──────────────────────────────────┘
         |
         v  POST /run_sse
```

```
Google Cloud Run
  ┌──────────────────────────────────────┐
  │  FastAPI Server (main.py)            │
  │  ┌──────────────────────────────┐    │
  │  │  Google ADK Agent Framework  │    │
  │  │  ┌──────────────┐            │    │
  │  │  │ root_agent   │ (Gemini 2.0 Flash) │
  │  │  │  ├─ vertex_agent (Vertex AI Search) │
  │  │  │  ├─ g2_agent (Google Search fallback) │
  │  │  │  ├─ get_brand_links()      │    │
  │  │  │  ├─ get_brand_ratings()    │    │
  │  │  │  ├─ get_sonary_categories() │    │
  │  │  │  ├─ find_matching_article() │    │
  │  │  │  └─ search_articles()      │    │
  │  │  └──────────────┘            │    │
  │  │  InMemorySessionService      │    │
  │  │  (sessions lost on restart)  │    │
  │  └──────────────────────────────┘    │
  └──────────────────────────────────────┘
         |                    |
         v                    v
  Google Vertex AI       Sonary WP API
  (LLM + Search)      (wp-staging.sonary.com)
  (429 throttling!)    (HTTP calls for brands,
                        articles, categories)
```

Current Technology Stack

Component	Technology	Location
Web Server	FastAPI + Uvicorn	Google Cloud Run
Agent Framework	Google ADK	Inside FastAPI server
LLM	Gemini 2.0 Flash	Google Vertex AI
Article Search	Vertex AI Search (datastore)	Google Cloud
Fallback Search	Google Search API	Google Cloud
Brand/Article Data	HTTP API calls to WordPress	Sonary WP site
Sessions (agent-side)	InMemorySessionService	Server RAM (volatile)
Sessions (app-side)	UsersAiSessions table	Sonary App MySQL (persistent)
Chat History	ChatHistory table	Sonary App MySQL (persistent)
Memory	None	N/A
Caching	Python dict (in-memory)	Server RAM (volatile)
Database	MySQL on RDS	AWS (unused by chatbot directly)

Current Data Flow (Single Request)

1. User sends message via Sonary Booster chat UI

2. Node.js `Ai.service.ts` saves user request to `ChatHistory` table
3. Node.js calls `POST /run_sse` on the Python FastAPI server (Cloud Run)
4. ADK `root_agent` receives the message with hardcoded `session_vsearch_1` session ID
5. Agent decides which tools to call:
   - `vertex_agent` (Vertex AI Search) for article queries
   - `get_brand_links()` makes HTTP call to `wp-staging.sonary.com` API
   - `get_brand_ratings()` makes HTTP call to WP API per brand
6. Agent combines tool results with LLM response (Gemini 2.0 Flash via Vertex AI)
7. Response streamed back via SSE
8. Node.js saves agent response to `ChatHistory` table

Current Caching

All caches are Python dictionaries stored in server RAM:

Cache	TTL	What It Stores	Lost On Restart?
`_brand_index`	1 hour	Brand name → {id, category, slug, logos, rating} mapping	Yes
`_brand_data_cache`	1 hour	Full brand data by brand_id	Yes
`_categories_cache`	1 hour	List of active category slugs	Yes
`_articles_cache`	1 hour	List of article slugs	Yes
`_rating_cache`	1 hour	Brand ratings by name:category	Yes

Problems With Current Architecture

#	Problem	Root Cause	Impact
1	**429 RESOURCE_EXHAUSTED**	Vertex AI PayGo throttling due to high global demand	Users get no response. Service is unreliable.
2	**No conversation context**	Hardcoded single session ID (`session_vsearch_1`) shared by all users	Agent treats every message as a new conversation.
3	**Sessions lost on restart**	`InMemorySessionService` stores in server RAM	Every deployment wipes all agent sessions.

4	**Slow responses**	Cross-cloud calls (Cloud Run → Vertex AI → WP API)	Multiple HTTP round-trips per request.
5	**Inaccurate responses**	Vertex AI Search returns fuzzy results; slug-matching for articles	Agent guesses brand info instead of precise lookups.
6	**No long-term memory**	No memory implementation	Agent can't remember user preferences across sessions.
7	**Fragile data pipeline**	Tools depend on WP API being up and fast	If WP site is slow/down, chatbot breaks.
8	**Expensive fix proposed**	Google suggested $2,000/month Provisioned Throughput	Not viable for current traffic volume (~116K tokens/day).

New Architecture (After)

```
Sonary Booster App (Frontend)
         |
         v
Sonary App Node.js (ECS) -- UNCHANGED
   ┌──────────────────────────────────────┐
   │  Ai.service.ts                       │
   │  - Manages sessions (UsersAiSessions)│
   │  - Saves chat history (ChatHistory)  │
   │  - Calls AI bot via HTTP POST        │
   │  - Streams responses to frontend     │
   └──────────────────────────────────────┘
         |
         v  POST /chat
AWS ECS Fargate (same VPC as RDS)
   ┌──────────────────────────────────────┐
   │  FastAPI Server (main.py)            │
   │                                      │
   │  ┌──────────────────────────────┐    │
   │  │  LangChain ReAct Agent       │    │
   │  │  (create_react_agent + Executor)  │
   │  │                              │    │
   │  │  LLM: ChatLiteLLMRouter      │    │
   │  │  ┌──────────────────────┐    │    │
   │  │  │ Primary: Bedrock Haiku 4.5 │    │
   │  │  │ Fallback: Anthropic Direct │    │
   │  │  └──────────────────────┘    │    │
   │  │                              │    │
   │  │  Tools (execute locally):    │    │
   │  │  ├─ get_brand_links()    → Redis/RDS │
   │  │  ├─ get_brand_ratings()  → Redis/RDS │
   │  │  ├─ get_sonary_categories() → Redis │
   │  │  └─ search_articles()    → Bedrock KB │
   │  │                              │    │
   │  │  Context: last 10 msgs from MySQL │
   │  └──────────────────────────────┘    │
   └──────────────────────────────────────┘
         |           |           |           |
         v           v           v           v
  LiteLLM Router  AWS RDS     ElastiCache  Bedrock KB
   ┌──────────┐   (MySQL)     (Redis)      ┌──────────┐
   │ Bedrock  │   Brand data  Brand cache  │ Articles │
   │ Anthropic│   Article     Category     │ (S3 sync)│
   │ Google AI│   Category    Rating cache │ Embeddings│
   └──────────┘   Chat hist.               └──────────┘
```

New Technology Stack

Component	Technology	Why
Web Server	FastAPI + Uvicorn	Same as current, minimal change
Agent Framework	LangChain (`create_react_agent` + `AgentExecutor`)	Multi-agent future, large ecosystem, provider-neutral
LLM Routing	LiteLLM Router via `ChatLiteLLMRouter`	Multi-provider fallback, cost tracking, auto rate-limit handling
Primary LLM	Claude 4.5 Haiku via AWS Bedrock	Best price/performance for the use case ($0.80/$4.00 per 1M tokens)
Fallback LLM	Claude 4.5 Haiku via Anthropic Direct API and Gemini 2.5 Flash	Same model, different provider. Avoids single-provider dependency
Article Search (RAG)	Bedrock Knowledge Base + `AmazonKnowledgeBasesRetriever`	Managed chunking, embedding, indexing. ~$1-2/month. LangChain built-in integration
Brand/Category Data	Direct SQL queries to RDS + Redis cache	Structured data, exact field lookup. Replaces WP API HTTP calls
Conversation Context	Load last 10 messages from Sonary App MySQL `ChatHistory` table	Already persisted by the Node.js service. No new storage needed
Long-term Memory	PostgreSQL table (Phase 2, post-launch)	Simple summaries. Low priority for launch
Caching	Redis on ElastiCache	Survives restarts, shared across ECS instances, built-in TTL
Deployment	ECS Fargate	AWS-native, Docker-based, same VPC as RDS and Redis
Article Sync	APScheduler cron inside ECS (daily)	Background job syncs articles from RDS to S3

		for Bedrock KB ingestion. No separate Lambda or EventBridge needed.

Component Deep Dive

A. LangChain Agent

Role: The orchestration layer that replaces both Google ADK and the previously proposed Bedrock Agent. Receives the user's message, decides which tools to call, executes them locally, and generates the final response.

How it works:

LangChain's `create_react_agent` creates a ReAct (Reason + Act) loop:

1. Agent receives user message + chat history
2. LLM reasons: "The user wants POS recommendations. I should look up brand data."
3. Agent calls `get_brand_links("Square", query_context="restaurant POS")`
4. Tool executes **locally** (Redis cache → RDS fallback) and returns brand card data
5. Agent observes the result, decides if more tools are needed
6. Agent calls `get_brand_links("Toast", ...)` for the next brand
7. Agent generates final response combining all tool results

```
from langchain.agents import create_react_agent, AgentExecutor
from langchain_litellm import ChatLiteLLMRouter
from langchain_core.prompts import ChatPromptTemplate,
MessagesPlaceholder

llm = ChatLiteLLMRouter(router=litellm_router)

prompt = ChatPromptTemplate.from_messages([
    ("system", SYSTEM_PROMPT),
    MessagesPlaceholder("chat_history"),
    ("human", "{input}"),
    MessagesPlaceholder("agent_scratchpad"),
])

agent = create_react_agent(llm=llm, tools=tools, prompt=prompt)
executor = AgentExecutor(
    agent=agent,
    tools=tools,
    max_iterations=5,
    handle_parsing_errors=True,
    verbose=False,
)

result = executor.invoke({
    "input": user_message,
    "chat_history": recent_messages,
})
```

Advantages over Bedrock Agent (v1.0 approach):

Aspect	Bedrock Agent (v1.0)	LangChain Agent (v2.0)
Tool execution	Return Control round-trip to AWS	Local in-process, no network hop

LLM provider	Locked to Bedrock	Any provider via LiteLLM
Rate limit handling	Bedrock's default limits (restrictive)	LiteLLM Router auto-fallback
Adding new tools	AWS Console or Terraform	Decorate a Python function with `@tool`
Prompt management	AWS Console text field	Python file in your codebase
Future multi-agent	Not supported	LangGraph upgrade path

Advantages over Google ADK (current):

Aspect	Google ADK (current)	LangChain Agent (v2.0)
LLM requirement	Must use Gemini models	Any LLM via LiteLLM
Provider lock-in	Google Cloud only	Provider-neutral
Session management	InMemorySessionService (volatile)	Stateless -- uses existing MySQL history
Ecosystem	Smaller, Google-focused	Largest AI framework ecosystem
Multi-agent	Sub-agents only	Full LangGraph support

B. LiteLLM Router

Role: A lightweight gateway between LangChain and LLM providers. Handles multi-provider routing, automatic fallback on errors, retry logic, and cost tracking.

Configuration:

```
from litellm import Router

litellm_router = Router(
    model_list=[
        {
            "model_name": "sonary-haiku",
            "litellm_params": {
                "model": "bedrock/anthropic.claude-3-5-haiku-20241022-v1:0",
                "aws_region_name": "us-east-1",
            },
        },
        {
            "model_name": "sonary-haiku",
            "litellm_params": {
                "model": "anthropic/claude-3-5-haiku-20241022",
            },
        },
    ],
    fallbacks=[{"sonary-haiku": ["sonary-haiku"]}],
```

```
    retry_after=5,
    allowed_fails=2,
    num_retries=2,
)
```

How fallback works:

```
Request comes in
    |
    v
Try Bedrock Claude 4.5 Haiku (primary)
    |
    ├── Success (200) → Return response
    |
    ├── Rate limited (429) → Retry once after 5s
    |       |
    |       ├── Success → Return response
    |       └── Still 429 → Fall back
    |
    └── Other error (500) → Fall back
            |
            v
Try Anthropic Direct Claude 4.5 Haiku (fallback)
    |
    ├── Success → Return response
    └── Fail → Return error to user
```

Both deployments use Claude 4.5 Haiku, so response quality is identical regardless of which provider serves the request.

Cost tracking: LiteLLM tracks token usage per provider, enabling cost monitoring via logs:

- Bedrock Haiku: $0.80 input / $4.00 output per 1M tokens
- Anthropic Direct Haiku: $0.80 input / $4.00 output per 1M tokens (same price)

C. Bedrock Knowledge Base (Article RAG)

Role: Provides semantic search over Sonary's article content. When a user asks "How to use gift cards to grow sales?", the KB finds the most relevant articles by meaning (not keyword matching), and the agent includes them in the response with links.

How data gets into the KB:

1. On server startup, APScheduler registers a daily cron job (2:00 AM UTC)
2. At 2:00 AM, the sync job runs as a background task inside the FastAPI server process
3. Reads published articles from RDS (`wp_posts` table)
4. Exports each article as a JSON file to S3 (`s3://sonary-kb-articles-{env}/articles/`)
5. Triggers Bedrock KB `StartIngestionJob` API
6. Bedrock automatically chunks the text, generates embeddings (Titan V2), and updates the vector index
7. The server continues handling chat requests normally during sync

What the KB replaces:

Before (current)	After (KB)
`vertex_agent` sub-agent (Vertex AI Search)	Bedrock KB semantic search
`g2_agent_tool` (Google Search fallback)	Not needed -- KB covers articles

`find_matching_article()` slug-matching	Semantic search (meaning-based)
`search_articles()` keyword matching	Semantic search (meaning-based)

What the KB does NOT replace:

- Brand data lookups (structured data -- handled by tools + DB)
- Category listings (structured data -- handled by tools + Redis)
- Rating calculations (computed data -- handled by tools)

LangChain integration:

```
from langchain_aws import AmazonKnowledgeBasesRetriever

retriever = AmazonKnowledgeBasesRetriever(
    knowledge_base_id="your-kb-id",
    retrieval_config={"vectorSearchConfiguration": {"numberOfResults": 3}},
)
```

D. Redis Cache (ElastiCache)

Role: Fast in-memory cache for structured data needed frequently during tool execution. Replaces the current Python dict caches that are lost on restart.

What's cached in Redis:

Cache Key Pattern	Data	TTL	Source
`brand:index`	Full brand name → {id, category, slug, logos, rating} mapping	1 hour	RDS
`brand:data:{brand_id}`	Complete brand data (outlinks, pricing, images, selling lines)	1 hour	RDS
`categories:active`	List of active category slugs	1 hour	RDS
`ratings:{brand_name}:{category}`	Pre-computed average ratings	1 hour	RDS

Why Redis instead of Python dicts (current approach):

- Survives server restarts and deployments
- Shared across multiple ECS task instances (horizontal scaling)
- Dedicated memory (doesn't compete with application heap)

- Built-in TTL expiration (no manual cleanup code)
- Atomic operations (thread-safe without locks)

E. RDS Database (Existing)

Role: The source of truth for all Sonary data. Tools query it directly instead of calling the WordPress HTTP API.

Data Needed	Current Source	New Source
Brand list (name, slug, category, logo)	`GET /all-products` WP API	`SELECT` from products/brands table
Brand details (outlinks, pricing, images)	`GET /brands?ids=X` WP API	`SELECT` from brand details table
Active categories	`GET /categories` WP API	`SELECT` from categories table
Article slugs (for sync)	`GET /articles-list` WP API	`SELECT` from `wp_posts` table
Chat history (for context)	Not used by chatbot	`SELECT` from `ChatHistory` table (Sonary App DB)

F. Conversation Context

How it works today: The Sonary App Node.js service (`Ai.service.ts`) already stores every message in the `ChatHistory` MySQL table with `sessionId` , `type` (request/response), and `content` . The chatbot Python service currently ignores this data.

How it works in the new architecture:

1. Node.js sends `POST /chat` with `{ message, session_id, user_id }`
2. FastAPI loads last 10 messages from `ChatHistory` table for that `session_id`
3. Messages are converted to LangChain format:

```
from langchain_core.messages import HumanMessage, AIMessage

def load_chat_history(session_id: str, limit: int = 10) -> list:
    rows = db.execute(
        "SELECT type, content FROM ChatHistory "
        "WHERE sessionId = %s ORDER BY id DESC LIMIT %s",
        [session_id, limit]
    )
    messages = []
    for row in reversed(rows):
        if row["type"] == "request":
            messages.append(HumanMessage(content=row["content"]))
        else:
            messages.append(AIMessage(content=row["content"]))
    return messages
```

4. Chat history is passed to the LangChain agent as `chat_history`
5. The agent uses this context to understand follow-up questions

No separate session storage needed. The Node.js app manages sessions (`UsersAiSessions` table) and persists chat history (`ChatHistory` table). The Python service is stateless.

Data Flow Diagrams

Flow 1: Simple Question (No Tools Needed)

Example: "What is a POS system?"

```
User
  |
  v
Node.js Ai.service.ts
  ├── Save user message to ChatHistory table
  ├── POST /chat {message, session_id, user_id}
  v
FastAPI Server (ECS)
  ├── Load last 10 messages from ChatHistory (MySQL)
  ├── Pass to LangChain AgentExecutor
  |     └── LLM (LiteLLM → Bedrock Haiku)
  |           └── Generates answer from training knowledge
  |               + agent instructions (no tools needed)
  ├── Return response
  v
Node.js Ai.service.ts
  ├── Save agent response to ChatHistory table
  └── Stream to frontend
```

Flow 2: Brand Recommendation (Tools + DB/Redis)

Example: "What's the best POS system for my restaurant?"

```
User
  |
  v
Node.js → POST /chat → FastAPI
  |
  v
Load chat_history from MySQL
  |
  v
LangChain Agent (ReAct loop):
  |
  ├ THINK: "User wants restaurant POS. I'll get ratings first."
  |
  ├ ACT: get_brand_ratings(["Square", "Toast", "Lightspeed"], "pos")
  |    └── Redis cache HIT? → return cached ratings
  |        Redis MISS? → Query RDS → cache in Redis → return
  |    RESULT: sorted_brands = ["Toast", "Square", "Lightspeed"]
  |
  ├ THINK: "Toast has highest rating. Get card data for top 3."
  |
  ├ ACT: get_brand_links("Toast", query_context="restaurant POS",
position=1)
  |    └── Redis → RDS → return brand card format
  |
  ├ ACT: get_brand_links("Square", ..., position=2)
  |    └── Redis → return (cached from ratings call)
  |
  ├ ACT: get_brand_links("Lightspeed", ..., position=3)
  |    └── Redis → RDS → return brand card
  |
  ├ THINK: "I have all 3 brand cards. Generate final response."
  |
  └ RESPOND: Formatted response with 3 brand cards
  |
  v
Node.js saves response → Stream to frontend
```

Flow 3: Article Search (Bedrock KB)

Example: "How to use gift cards to grow sales?"

```
User
  |
  v
Node.js → POST /chat → FastAPI
  |
```

```
  v
Load chat_history → LangChain Agent:
  |
  ├─ THINK: "User asking about gift card strategies. Search articles."
  |
  ├─ ACT: search_articles("how to use gift cards to grow sales")
  |     └── AmazonKnowledgeBasesRetriever.invoke(query)
  |          └── Bedrock KB searches embeddings
  |               └── Returns: top 3 matching article passages + metadata
  |     RESULT: [{title, url, content_snippet}, ...]
  |
  ├─ THINK: "Found relevant article. Include URL in response."
  |
  └─ RESPOND: Summary of article + link to sonary.com/content/...
  |
  v
Node.js saves response → Stream to frontend
```

Flow 4: Article Sync Pipeline (Background, Daily)

```
ECS Fargate (FastAPI server -- always running)
  |
  |  On startup: APScheduler registers daily cron job at 2:00 AM UTC
  |
  |  At 2:00 AM UTC, background job triggers:
  |
  ├── Connect to RDS (via Secrets Manager)
  ├── SELECT published articles from wp_posts
  ├── For each article:
  |     ├── Strip HTML tags from content
  |     └── Upload JSON to S3: s3://sonary-kb-articles-{env}/articles/article-{id}.json
  |         {title, content, url, slug, published_date}
  ├── Call Bedrock: StartIngestionJob(knowledgeBaseId, dataSourceId)
  └── Log: "Synced N articles, ingestion job started"
  |
  |  Server continues handling chat requests normally
  |
        v (async, ~5-15 min)
  Bedrock KB:
    - Chunks article text
    - Generates embeddings (Titan V2)
    - Updates vector index
    - Ready for agent queries
```

Flow 5: Follow-up Question (Context from Chat History)

Example: (Continuing from Flow 2) "Tell me more about the first one"

```
User: "Tell me more about the first one"
  |
  v
Node.js → POST /chat → FastAPI
  |
  v
Load chat_history from MySQL:
  [HumanMessage("best POS for restaurant"),
   AIMessage("Here are top 3: 1. Toast, 2. Square, 3. Lightspeed..."),
   HumanMessage("Tell me more about the first one")]
  |
  v
LangChain Agent:
  ├─ THINK: "From chat history, 'the first one' = Toast. Get detailed info."
  ├─ ACT: get_brand_links("Toast", query_context="restaurant POS details")
  └─ RESPOND: Detailed Toast information with brand card
```

Caching Strategy

Cache Architecture

```
Tool receives request (e.g., get_brand_links("Square"))
        |
        v
```

```
Check Redis cache ──> HIT (data < TTL) ──> Return cached data
        |                                        (< 1ms latency)
        | MISS
        v
Query RDS database ──> Get fresh data ──> Store in Redis with TTL
        |                                        (10-50ms latency)
        v
Return fresh data
```

Lazy Initialization (No Cache Warming Needed)

Caches populate on first use. When a tool needs data and Redis has no cached value (TTL expired), the tool queries RDS, stores the result in Redis, and returns it. This adds ~300ms to the first request after a TTL expiry.

Why no cache warming: ElastiCache Redis is an independent service that survives ECS restarts and deployments. With a 1-hour TTL, caches refresh naturally. The only scenario requiring a full cache rebuild is a Redis restart, which is rare with managed ElastiCache.

Cache Invalidation

Strategy	When	How
TTL expiration	After 1 hour	Redis auto-expires the key. Next request rebuilds from RDS.
Manual flush	After bulk data update	Admin API endpoint: `POST /admin/cache/flush`

Cache Failure Handling

If Redis is down or unreachable:

- Tools fall through to direct RDS queries (slower but functional)
- Redis client has error handling that returns `None` on connection failure
- Application continues serving requests without caching

Session and Memory Management

Sessions

Aspect	Details
Owner	Sonary App Node.js service (existing)
Storage	`UsersAiSessions` table in Sonary App MySQL (user-service DB)
Chat History	`ChatHistory` table in Sonary App MySQL (user-service DB)
Session ID	UUID generated by Node.js, passed to Python service
User ID	From authenticated user, passed to Python service

Python service role	Stateless. Loads recent history on each request, responds, done.

What this means: The Python AI chatbot service has **no session storage of its own**. It receives `session_id` and `user_id` from the Node.js app, loads chat history from MySQL, processes the request, and returns a response. Sessions survive restarts because they're stored in MySQL by the Node.js app.

Long-term Memory (Phase 2 -- Post-Launch)

After the core migration is stable, add a simple `agent_memories` table in PostgreSQL:

Column	Type	Description
`id`	SERIAL	Primary key
`user_id`	VARCHAR	User identifier
`summary`	TEXT	Summarized memory (e.g., "User runs a restaurant, interested in POS and payment systems")
`created_at`	TIMESTAMP	When the memory was created
`updated_at`	TIMESTAMP	Last update

On each request, load the user's memory summary and prepend it to the system prompt. After every N conversations, use the LLM to generate an updated summary.

This is simple, effective, and avoids the complexity of vector-based memory stores.

Tool Definitions

All tools are Python functions decorated with LangChain's `@tool` decorator. The agent calls them based on the user's question and the tool descriptions.

get_brand_links

```
@tool
def get_brand_links(
    brand_name: str,
    category: str = "",
    query_context: str = "",
    position: int = 1
) -> str:
    """Get brand card data including logo, description, pricing, and visit link
    for a specific brand. Returns a pre-formatted brand card string.
    Call this for EVERY brand you mention in your response."""
    # 1. Load brand index from Redis (or build from RDS)
    # 2. Find brand with fuzzy matching
    # 3. Select best category instance (rating + context-based)
    # 4. Fetch full brand data from Redis/RDS
    # 5. Check not_paying flag
    # 6. Format 4-line brand card
    # Returns: formatted card string or error message
```

get_brand_ratings

```
@tool
def get_brand_ratings(brand_names: str, category: str = "") -> str:
    """Get ratings for multiple brands to determine display order.
    Returns brands sorted by rating (highest first).
    Call this BEFORE get_brand_links when comparing multiple brands.
    brand_names should be comma-separated:
'Square,Toast,Lightspeed'"""
    # 1. Parse comma-separated brand names
    # 2. Look up each brand in index
    # 3. Calculate average rating per brand
    # 4. Sort by rating descending
    # Returns: sorted brand list with ratings
```

get_sonary_categories

```
@tool
def get_sonary_categories() -> str:
    """Get all active business software categories from Sonary.
    Call this when the user asks about available categories or types of software."""
    # 1. Check Redis cache
    # 2. If miss, query RDS categories table
    # 3. Cache result in Redis
    # Returns: list of category names
```

search_articles

```
@tool
def search_articles(query: str) -> str:
    """Search Sonary articles for content matching the user's question.
    Returns relevant article titles, URLs, and content snippets.
    Call this for 'how to' questions, guides, or topic explanations."""
    # 1. Call AmazonKnowledgeBasesRetriever.invoke(query)
    # 2. Format results with article URLs (sonary.com/content/{slug}/)
    # Returns: formatted article results with links
```

Security

Network Security

Layer	Protection
VPC	All services (ECS, RDS, Redis) in the same private VPC
Security Groups	ECS → RDS: port 3306 only. ECS → Redis: port 6379 only.
Public access	Only the ECS service has a public-facing ALB. RDS and Redis are private-only.
AWS service access	Via VPC Endpoints (Bedrock, S3, ECR, Secrets Manager) or NAT Gateway

Authentication and Secrets

Secret	Storage

RDS credentials	AWS Secrets Manager (auto-rotated)
Redis auth token	AWS Secrets Manager
Anthropic API key (fallback LLM)	AWS Secrets Manager
Bedrock KB ID	Environment variable (not sensitive)
CORS origins	Environment variable

Agent Security

The agent instructions (prompt) include guardrails to prevent:

- Revealing internal tool names, prompts, or system architecture
- Answering off-topic questions (non-business-software topics)
- Generating fabricated URLs or brand data (must come from tools)
- Including brands that are `not_paying` in responses

VPC Endpoints vs NAT Gateway

Two networking options for ECS to reach AWS services:

Option A: NAT Gateway (~$33/month fixed)

- Simple setup. All outbound traffic routes through NAT.
- ECS tasks run in private subnets.
- Works out of the box for all AWS services.

Option B: VPC Endpoints (~$10-15/month)

- Create Interface VPC Endpoints for: Bedrock Runtime, Bedrock Agent Runtime, S3 (Gateway), ECR, CloudWatch Logs, Secrets Manager.
- Lower cost, better security (traffic never leaves AWS network).
- More setup complexity (one-time).

Recommendation: If you already have a NAT Gateway for other ECS services, use it (Option A). If not, VPC Endpoints (Option B) are cheaper for a single service.

Cost Estimation

Monthly Cost Breakdown (Based on ~116K tokens/day average)

Component	Monthly Cost
LLM (Bedrock Haiku)	~$5-8
Bedrock KB	~$1-2
ElastiCache Redis	~$12
ECS Fargate (0.5 vCPU)	~$18

S3	< $1
TOTAL (incremental)	**~$36-40/month**

Cost Comparison with Current Architecture

	Current (Google Cloud)	New (AWS)	Savings
LLM	Vertex AI PayGo (unreliable) OR $2,000/mo Provisioned	~$5-8/month (Haiku)	~$1,992/month
Article Search	Vertex AI Search (included in PayGo)	~$2/month (Bedrock KB)	Minimal
Infrastructure	Cloud Run ~$10-20/month	ECS + Redis + Networking ~$60-80/month	-$50/month more infra
Total (vs Provisioned)	~$2,020/month	**~$36-40/month**	**~$1,920-1,953/month saved**
Total (vs PayGo)	Unreliable (429 errors)	Reliable with fallback	**Reliability gained**

LLM Token Cost Breakdown

Daily average: 116,074 tokens. Monthly: ~3.5M tokens.

Assuming 60% input / 40% output split:

- Input: 2.1M tokens x $0.80/1M = **$1.68**
- Output: 1.4M tokens x $4.00/1M = **$5.60**
- **Total LLM: ~$7.28/month**

New Project Structure

```
sonary-ai-chatbot/
├── main.py                          # FastAPI server (rewritten)
├── requirements.txt                 # Updated dependencies
├── Dockerfile                       # Updated for AWS
├── docker-compose.yml               # Local dev (app + Redis)
├── .env.example                     # Environment variables template
│
├── app/
│   ├── __init__.py
│   ├── agent/
│   │   ├── __init__.py
│   │   ├── langchain_agent.py       # LangChain agent + AgentExecutor setup
│   │   └── llm_config.py            # LiteLLM Router configuration
│   ├── tools/
│   │   ├── __init__.py
```

```
│   │   ├── brand_tool.py            # get_brand_links, get_brand_ratings (DB + Redis)
│   │   ├── article_tool.py          # search_articles (Bedrock KB retriever)
│   │   └── category_tool.py         # get_sonary_categories (DB + Redis)
│   ├── cache/
│   │   ├── __init__.py
│   │   └── redis_client.py          # Redis cache abstraction layer
│   ├── db/
│   │   ├── __init__.py
│   │   └── connection.py            # RDS connection pool + chat history loader
│   └── scheduler.py                 # APScheduler cron (article sync daily at 02:00 UTC)
│
├── config/
│   ├── __init__.py
│   ├── settings.py                  # Centralized configuration (env vars, constants)
│   └── prompts.py                   # Agent system prompt (adapted from sonary/prompt.py)
│
├── scripts/
│   └── sync_articles_to_s3.py       # Article sync module (called by APScheduler cron inside ECS)
│
├── docs/
│   ├── ARCHITECTURE.md              # v1.0 -- Bedrock Agent approach (reference)
│   ├── ARCHITECTURE_V2.md           # v2.0 -- This document (LangChain + LiteLLM)
│   ├── IMPLEMENTATION_PLAN.md       # v1.0 -- Bedrock Agent plan (reference)
│   └── IMPLEMENTATION_PLAN_V2.md    # v2.0 -- New implementation plan
│
├── tests/
│   ├── test_brand_tool.py
│   ├── test_article_tool.py
│   └── test_agent.py
│
├── .github/
│   └── workflows/
│       └── deploy-ecs.yml           # CI/CD: build -> ECR -> ECS
│
└── sonary/                          # OLD CODE (kept during migration, removed after cutover)
    ├── agent.py
    ├── prompt.py
    ├── brand_links_tool.py
    └── articles_tool.py
```

Frontend Integration Changes

Current API Contract (Node.js → Python)

The Sonary App Node.js service (`Ai.service.ts`) currently calls two endpoints:

1. **Session creation:**

```
POST {AI_BASE_URL}/apps/sonary/users/{userId}/sessions/{sessionId}
Body: { appName: "sonary", userId, sessionId }
```

2. **Chat message (SSE):**

```
POST {AI_BASE_URL}/run_sse
Body: {
  appName: "sonary",
  userId: "user-uuid",
  sessionId: "session-uuid",
  newMessage: {
    role: "user",
    parts: [{ text: "message text" }]
  }
}
```

New API Contract

Session creation: REMOVED -- sessions are fully managed by the Node.js app. No need to create sessions in the Python service.

Chat message:

```
POST {AI_BASE_URL}/chat
Body: {
  message: "message text",
  session_id: "session-uuid",
  user_id: "user-uuid"
}
Response: {
  response: "agent response text",
  session_id: "session-uuid"
}
```

Changes Required in Node.js Service

1. Update `AI_RUN_SSE_ENDPOINT` constant from `/run_sse` to `/chat`
2. Simplify request body format (flat JSON instead of nested `newMessage`)
3. Remove `createSessionInAiBot()` method (no longer needed)
4. Update `sendRequestToAiBot()` to use new request/response format
5. Handle both SSE streaming (Phase 2) and JSON response (Phase 1)

Environment Variables

```
# AWS Region
AWS_REGION=us-east-1

# LiteLLM Configuration
LITELLM_PRIMARY_MODEL=bedrock/anthropic.claude-3-5-haiku-20241022-v1:0
LITELLM_FALLBACK_MODEL=anthropic/claude-3-5-haiku-20241022

# Anthropic Direct API (fallback provider)
ANTHROPIC_API_KEY=sk-ant-...  # Or use Secrets Manager

# Bedrock Knowledge Base
BEDROCK_KB_ID=your-knowledge-base-id

# S3 (for article sync)
KB_S3_BUCKET=sonary-kb-articles-prod
BEDROCK_KB_DATASOURCE_ID=your-datasource-id

# Database (Sonary App MySQL -- for chat history)
SONARY_APP_DB_SECRET=sonary/app-db-credentials

# Database (WordPress MySQL -- for brand/article data)
WP_DB_SECRET=sonary/wp-rds-credentials

# Redis
REDIS_SECRET=sonary/redis-credentials

# Application
PORT=8080
ENVIRONMENT=production
LOG_LEVEL=INFO
CORS_ORIGINS=https://app.sonary.com,https://sonary.com

# Agent Configuration
MAX_AGENT_ITERATIONS=5
CHAT_HISTORY_LIMIT=10
CACHE_TTL_SECONDS=3600
```

Rollback Strategy

During Migration (Cloud Run runs in parallel)

```
Frontend (Node.js)
    |
    ├── AI_BASE_URL = new ECS endpoint ──> New architecture (LangChain)
    |
    └── (rollback) AI_BASE_URL = old Cloud Run endpoint ──> Old architecture (ADK)
```

The current Google Cloud Run service remains running throughout the entire migration. If issues arise with the new architecture, switch `AI_BASE_URL` back to the Cloud Run endpoint. This is an instant rollback (environment variable change + restart Node.js).

Component-level Rollback

If This Fails	Rollback
Bedrock LLM (429)	LiteLLM auto-falls back to Anthropic Direct. No action needed.
Bedrock KB returns no results	Agent responds from LLM knowledge only (degraded but functional). Fix sync and re-index.
RDS queries fail	Redis cache serves data for up to 1 hour (cache TTL). Fix DB connection.
Redis fails	Tools fall through to direct RDS queries (slower but functional).
Article sync fails	Bedrock KB retains previous index. Articles from last sync remain searchable. Fix sync code and restart ECS, or run sync manually: `python scripts/sync_articles_to_s3.py`.
Entire new service down	Switch `AI_BASE_URL` to Cloud Run (instant rollback).

Decommission Cloud Run

Only after 1-2 weeks of stable operation on the new AWS architecture:

1. Verify zero errors in CloudWatch logs
2. Verify response times are acceptable
3. Verify all tool outputs match expected format
4. Turn off Cloud Run service
5. Remove Google Cloud dependencies from project

Boto3 - usage

```
# boto3 Usage
```

```

| Location | Client | Operations |
|----------|--------|------------|
| `sonary_booster_agent/services/article_sync_service.py` | `boto3.client("s3")` | `put_object`, `list_objects_v2` (paginator), `delete_objects` |
| `sonary_booster_agent/services/article_sync_service.py` | `boto3.client("bedrock-agent")` | `start_ingestion_job` |
| `sonary_booster_agent/tools/article_tools.py` | `boto3.client("bedrock-agent-runtime")` | `retrieve` |
| `sonary_booster_agent/controllers/sonary_booster_controller.py` | `boto3.client("bedrock-agent")` | `get_knowledge_base` |

---

## What each does under the hood

**S3 (`article_sync_service.py`)**
- **`list_objects_v2`** — Lists objects under the articles prefix to find which article IDs exist in the bucket.
- **`delete_objects`** — Deletes objects for articles that were removed from the source (stale cleanup).
- **`put_object`** — Writes each article as a `.txt` file and a `.txt.metadata.json` file, and uploads a `manifest.json` index. The Knowledge Base is configured to use this bucket as its data source.

**bedrock-agent — `start_ingestion_job` (`article_sync_service.py`)**
After uploading (or updating) article files to S3, the service calls this to start a Bedrock Knowledge Base ingestion job. Bedrock then reads from the S3 data source, chunks and embeds the content, and updates the vector index so new or changed articles become searchable.

**bedrock-agent-runtime — `retrieve` (`article_tools.py`)**
The article search tool sends the user's query to the Knowledge Base. Bedrock runs vector search over the embedded articles, returns the best-matching chunks with metadata (e.g. article ID, link), and the tool formats them as article links/snippets for the chat response.

**bedrock-agent — `get_knowledge_base` (`sonary_booster_controller.py`)**
The health/connectivity check calls this to fetch the Knowledge Base by ID. It verifies credentials and that the KB exists and is reachable; the response includes status and name.

```

Future Roadmap

Phase 2 Enhancements (After Stable Launch)

Feature	Description	Effort
SSE Streaming	Stream agent responses token-by-token to the frontend	1-2 days
Long-term Memory	PostgreSQL `agent_memories` table with user summaries	2-3 days
pgvector Migration	Replace Bedrock KB with pgvector on PostgreSQL (after Payload CMS migration)	3-5 days

Prompt Caching	Use Bedrock's prompt caching for system prompt ($0.08/1M vs $0.80/1M)	1 day

Phase 3 Enhancements (Multi-Agent Future)

Feature	Description	Effort
LangGraph	Upgrade from `create_react_agent` to LangGraph for complex multi-step workflows	3-5 days
Additional Agents	New specialized agents (e.g., onboarding assistant, analytics helper)	Per agent
Shared Tool Registry	Centralized tool definitions used by multiple agents	2-3 days
Observability	LangSmith integration for agent tracing, evaluation, and monitoring	1-2 days

---

Document Version: 2.0 Last Updated: February 18, 2026 Previous Version: ARCHITECTURE.md (Bedrock Agent approach)